



PDF Download
3695053.3731100.pdf
31 January 2026
Total Citations: 0
Total Downloads: 8695

Latest updates: <https://dl.acm.org/doi/10.1145/3695053.3731100>

RESEARCH-ARTICLE

Avant-Garde: Empowering GPUs with Scaled Numeric Formats

MINSEONG GIL, Korea University, Seoul, South Korea

DONGHO HA

SIMLA BURCU HARMA, EPFL, Lausanne, Switzerland

MYUNG-KUK YOON, Ewha Womans University, Seoul, Seoul, South Korea

BABAK FALSAFI, EPFL, Lausanne, Switzerland

WONWOO RO, Yonsei University, Seoul, South Korea

[View all](#)

[Open Access Support](#) provided by:

[Ewha Womans University](#)

[Korea University](#)

[EPFL](#)

[Yonsei University](#)

Published: 21 June 2025

[Citation in BibTeX format](#)

ISCA '25: Proceedings of the 52nd Annual
International Symposium on Computer
Architecture

June 21 - 25, 2025
Tokyo, Japan

Conference Sponsors:
[SIGARCH](#)

Avant-Garde: Empowering GPUs with Scaled Numeric Formats

Minseong Gil*

Korea University
Seoul, Republic of Korea
ms7859@korea.ac.kr

Myung Kuk Yoon

Ewha Womans University
Seoul, Republic of Korea
myungkuk.yoon@ewha.ac.kr

Dongho Ha*

MangoBoost Inc.
Seoul, Republic of Korea
dongho.ha@mangoboost.io

Babak Falsafi

EcoCloud, EPFL
Lausanne, Vaud, Switzerland
babak.falsafi@epfl.ch

Simla Burcu Harma

EcoCloud, EPFL
Lausanne, Vaud, Switzerland
simla.harma@epfl.ch

Won Woo Ro

Yonsei University
Seoul, Republic of Korea
wro@yonsei.ac.kr

Yunho Oh

Korea University
Seoul, Republic of Korea
yunho_oh@korea.ac.kr

Abstract

The escalating computational and memory demands of deep neural networks have outpaced chip density improvements, making arithmetic density a key bottleneck for GPUs. Scaled numeric formats, such as FP8 and Microscaling (MX), improve arithmetic density by applying adaptive scaling factors across varying block sizes and multiple scaling hierarchies. Unfortunately, supporting diverse scaled numeric formats often requires GPUs to rely on software-based implementations, increasing instruction and register overhead and degrading performance. We propose Avant-Garde, a GPU microarchitecture that natively supports diverse scaled numeric formats by converting them into a consistent single-level internal representation. Avant-Garde integrates an Operand Transformer, a hardware module that dynamically flattens multi-level scaling formats into single-level internal representations, a novel Tensor Core, and an optimized data layout to eliminate instruction and register overhead. Our evaluations show that Avant-Garde achieves up to 74% higher throughput and 44% lower execution time, while maintaining accuracy within 0.2% compared to conventional GPUs.

CCS Concepts

• Computer systems organization → Parallel architectures.

Keywords

GPU, Deep Neural Network, Scaled Numeric Format

ACM Reference Format:

Minseong Gil, Dongho Ha, Simla Burcu Harma, Myung Kuk Yoon, Babak Falsafi, Won Woo Ro, and Yunho Oh. 2025. Avant-Garde: Empowering GPUs with Scaled Numeric Formats. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3695053.3731100>

*Minseong Gil and Dongho Ha contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License. ISCA '25, Tokyo, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1261-6/25/06

<https://doi.org/10.1145/3695053.3731100>

1 Introduction

Deep Neural Networks (DNNs) are widely used across diverse market segments, driven by an unprecedented growth in model and dataset sizes. For instance, GPT-3 contains 175 billion parameters and requires over 3×10^{23} arithmetic operations for training [10]. The sizes of the state-of-the-art models keep increasing exponentially every year [35]. This rapid growth has outpaced improvements in chip density [40], leading to significant computational and memory challenges for both training and inference. The energy consumption and environmental impact of both training and using these models have also become significant concerns.

Scaled numeric formats have emerged as a promising solution to improve arithmetic density (operations per second per mm^2) while maintaining accuracy in large-scale DNN workloads [8, 9, 12, 14, 25, 41, 42]. These formats represent numeric values by grouping them into recursive sets called *blocks*, each associated with a shared scaling factor. The scaling factor adjusts the dynamic range uniformly for all values within the same block, reducing quantization error and enhancing computational efficiency. Blocks can be organized hierarchically into a single or multiple *levels*. For instance, a single-level scaled numeric format applies a single scaling factor uniformly across all elements within a block. In contrast, multi-level scaled numeric formats divide each block further into smaller subsets, each with its own scaling factor. At the lowest level, each element is represented in floating or fixed point. The hierarchy, as such, enables precision adjustments across levels and provides fine-grained control over data representation. FP8 and Microscaling (MX) formats apply this approach to balance memory efficiency and computational accuracy for large-scale DNN workloads.

Unfortunately, modern GPUs rely mostly on software to support scaled numeric formats. Recent NVIDIA GPUs only support FP8 which relies on per-tensor scaling factors implemented directly in software [30]. Emerging MX formats on current GPUs similarly require each operand to be converted in software from the scaled numeric format to a fixed-point representation, applying the scaling factors to each element at each level to prepare them for an arithmetic operation (e.g., matrix multiplication and accumulation). This software overhead is simply prohibitive precluding the use of arbitrary scaled numeric formats in GPUs today.

In this paper, we make the observation that for GPUs to natively support scaled numeric formats, they must enable an operand representation in microarchitecture that lends itself well to efficient arithmetic and operand storage in hardware. Hardware can implement operand conversion from scaled numeric formats to this representation to minimize the conversion overhead. Moreover, operands can remain in this representation for the duration of a workload’s execution to minimize the conversion cadence. Despite variations in block size and the number of levels in scaled numeric formats, the values can be converted to a single-level representation used for computation. Fortunately, arithmetic with a single-level representation can be readily implemented in microarchitecture [8, 26, 50] supporting all arithmetic operations required to execute DNN workloads. Therefore, a GPU that natively supports scaled numeric formats can virtually eliminate the software overhead incurred to use them.

We propose Avant-Garde, a novel GPU microarchitecture, which is to the best of our knowledge, the first of its kind to natively support scaled numeric formats. The core innovation of Avant-Garde lies in “flattening” operands in hardware from scaled numeric formats into a single-level format for computation. We call a block transformed from a scaled numeric format to a single-level one a *flattened block*. Avant-Garde microarchitecture incorporates key components such as Operand Transformer, a specialized module that transforms operands from a multi-level format into flattened blocks, and a redesigned Tensor Core architecture that performs seamless arithmetic operations using a flattened format. Avant-Garde also employs an optimized data layout and storage mechanism that manages scaling factors and numeric values in memory efficiently without altering their original format.

Our evaluation of Avant-Garde on DNN workloads employing scaled numeric formats (i.e., MX and Hybrid Block Floating Point (HBFP)) demonstrates significant improvements in performance and energy efficiency over a baseline GPU architecture. Avant-Garde achieves a 74% improvement in throughput compared to baseline GPUs that support FP8 only. Also, Avant-Garde reduces inference execution time by 44%.

This paper makes the following contributions:

- We analyze the bottlenecks in supporting scaled numeric formats in GPUs and reveal inefficiencies caused by the lack of native support for dynamic scaling. Standard Tensor Cores do not support multi-level scaling factors, leaving software-based alternatives as the only option. This results in increased instruction overhead and register usage, which in turn reduces arithmetic density.
- We propose Avant-Garde, a novel GPU microarchitecture that efficiently handles scaled numeric formats. Avant-Garde integrates an Operand Transformer, a Avant-Garde Tensor Core architecture, and an optimized data layout to improve arithmetic density.
- We evaluate Avant-Garde on MX-compatible formats across a range of DNN workloads. Avant-Garde achieves a 74% increase in throughput and a 44% reduction in inference time compared to the baseline.

- We demonstrate using ViT-Base, BERT, and GPT-2 that flattened MX9 deviates by less than 0.2% in accuracy or perplexity compared to both original MX9 and FP32.

The remainder of this paper is organized as follows. Section 2 explains why GPUs need a native support for scaled numeric formats and our insights. Section 3 introduces the Avant-Garde architecture. Section 4 and Section 5 present the methodology and the experimental results, respectively. Section 6 discusses related work. Finally, Section 7 concludes this paper.

2 Why Avant-Garde?

In this section, we first explain the fundamentals of scaled numeric formats and the operations they involve. We then describe the challenges of executing these operations efficiently on GPUs. Finally, we present our key insights for overcoming these challenges.

2.1 The Rise of Scaled Numeric Formats

Arithmetic density has emerged as a major bottleneck in platforms designed for DNN workloads, posing limitations on computational efficiency and scalability [13, 19, 32]. The rapid growth in DNN model and dataset sizes has led to substantial electrical consumption, raising both environmental and operational concerns across industries. Modern large language models, such as GPT-3 with 175 billion parameters, require over 3×10^{23} arithmetic operations [10], creating unprecedented computational demands. As Moore’s Law slows down and approaches its limits, maximizing arithmetic density in GPUs has become an enduring challenge for chip designers.

Quantization and mixed-precision methods have been widely explored to improve the efficiency of DNN workloads, especially under the constraints of current GPUs. Prior work has proposed various quantization methods, which are particularly beneficial for inference tasks by lowering precision to decrease computational load and data traffic [1, 48, 52]. However, these methods are constrained by the numeric formats supported in GPUs. Similarly, mixed-precision methods have been developed to reduce computational and memory requirements [17, 28, 36, 51]. While effective, these methods often rely on floating-point formats to maintain accuracy, which limits the achievable throughput compared to fixed-point formats. As a result, these approaches fall short of fully leveraging GPU efficiency, highlighting the need for a solution that can optimize both arithmetic density and memory overhead in training and inference.

Scaled numeric formats have emerged as a promising solution for improving arithmetic density and memory overhead while maintaining sufficient accuracy in both training and inference. These formats are characterized by two core concepts: blocks and levels. Blocks group multiple elements under a shared scaling factor, reducing the need for per-element scaling and improving memory efficiency. This block-based format enables coarse-grained precision management, which is particularly effective for DNN workloads. Levels define the hierarchical depth of scaling within blocks, representing how scaling factors are organized. Single-level formats apply a uniform scaling factor across all elements in a block, offering a simple and efficient mechanism for precision adjustment. In contrast, multi-level formats introduce additional scaling factors at finer granularities, such as subsets of elements, to enable more

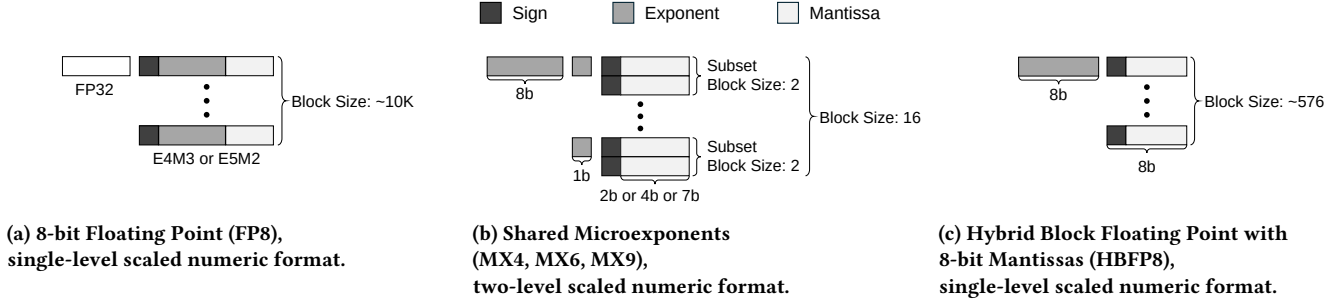


Figure 1: Emerging scaled numeric formats. The 8-bit floating-point format consists of two encodings: one with a 4-bit exponent and a 3-bit mantissa (E4M3), and another with a 5-bit exponent and a 2-bit mantissa (E5M2) [30]. The shared micro exponent format offers three configurations: MX4, MX6, and MX9, with mantissa sizes of 2 bits, 4 bits, and 7 bits, respectively [4].

precise control over data representation. For example, two-level formats apply a secondary scaling factor within each subset, providing finer-grained control over precision within a block.

Figure 1 shows examples of emerging scaled numeric formats. Recent NVIDIA GPUs support an 8-bit floating-point format (FP8), optimized for DNN workloads (Figure 1a) [30]. FP8 incorporates a per-tensor scaling factor to mitigate the limited dynamic range of 8-bit representations. This per-tensor scaling enables the use of low-bitwidth operations, which increases arithmetic density by reducing memory and compute requirements. However, FP8 relies on software to implement and manage these scaling factors, which might limit GPU performance.

Microsoft has introduced an alternative numeric format called shared microexponents (MX) [4, 42], which has been made available through the Open Compute Project (OCP) [2]. The MX format (Figure 1b) offers three configurations (MX4, MX6, and MX9) depending on the bitwidth allocated per element. These formats commonly use two-level scaling: a set of 16 elements shares a first-level scaling factor as a block, while each subset of two elements within the block forms a second-level block sharing a second-level scaling factor. Among the MX formats, MX9, which includes a 7-bit mantissa, is the only configuration designed to achieve training and inference accuracy comparable to traditional formats. Its peak arithmetic throughput is equivalent to that of a 7-bit fixed-point format.

Hybrid block floating point (HBFP), which uses a shared scaling factor across a block of numbers, improves arithmetic density and reduces memory usage (Figure 1c) [8, 9]. HBFP is functionally equivalent to MXINT but employs a larger block size (576 instead of 32) [2]. Recent advancements have introduced HBFP variants with 6-bit and 4-bit mantissas, extending beyond the original 8-bit configuration. These lower-precision HBFP formats enable energy-efficient computation for large-scale model training and inference, addressing the growing demand for scalable and power-efficient deep learning solutions [14].

2.2 Challenges with Scaled Numeric Formats

Native GPU support for scaled numeric formats beyond FP8 is essential to fully leverage their computational efficiency benefits. Software stacks for quantization methods tailored to MX formats have been developed to address the limitations of native FP8 [15, 16].

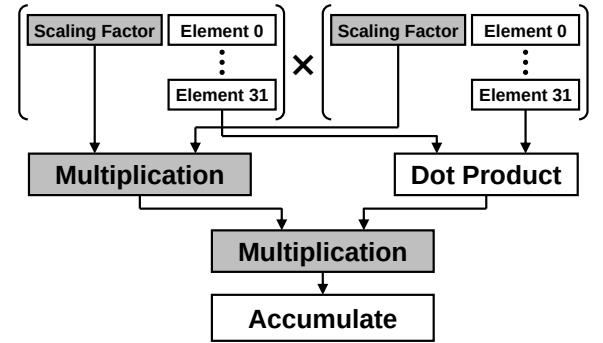


Figure 2: MMA operations with scaled numeric formats with conventional GPUs. Blocks highlighted in grey represent operations that Tensor Core cannot process natively. These operations involve calculating and applying a scaling factor to the product result before accumulation.

These methods mitigate precision loss and improve numerical robustness, making them more effective for low-bitwidth computation. Also, several post-training quantization methods that employ MX formats have been introduced to enhance inference accuracy [18, 24, 39, 43]. For training, an energy-efficient approach based on HBFP has been introduced to address challenges related to dynamic range limitations and precision degradation [14]. These developments indicate that DNN models can achieve improved efficiency and accuracy by adopting various scaled numeric formats beyond FP8. While these formats can be quantized to FP8 for execution on GPUs, such conversion may compromise their superior arithmetic density or accuracy.

Figure 2 illustrates how computation with scaled numeric formats adapts to different scaling hierarchies. The execution flow is determined by the scaling hierarchy and precision of the target scaled numeric format. In a single-level format, the GPU computes dot products directly on the elements and multiplies the scaling factors from each input block separately. The GPU then multiplies the dot product result by the combined scaling factor to produce the final output. In a multi-level format, the GPU first applies all scaling factors, except that of the first-level block, to the corresponding

```

mma.load.a {R0-1}, [RD0], R21
mma.load.b {R2-3}, [RD1], R22
mma.mma {R0-7}, {R0-1}, {R2-3}, {R23}
ld.global R16, [RD2];
ld.global R17, [RD2+0x4];
ld.global R18, [RD3];
ld.global R19, [RD3+0x4];
mul R20, R16, R18
mad R8, R0, R20, R8
mad R9, R1, R20, R9
mul R0, R17, R18
mad R10, R2, R0, R10
mad R11, R3, R0, R11
mul R0, R16, R19
mad R12, R4, R0, R12
mad R13, R5, R0, R13
mul R1, R18, R19
mad R14, R6, R1, R14
mad R15, R7, R1, R15

```

Figure 3: CUDA instruction stream for Matrix Multiply-Accumulate (MMA) operations using scaled numeric formats on conventional GPUs.

elements. This adjustment transforms the data layout into a form equivalent to a single-level format. After the adjustment, the GPU computes dot products over the modified elements and multiplies the accumulated result by the first-level scaling factor.

Implementing scaled numeric formats beyond FP8 in GPUs poses a significant challenge. These formats rely on shared scaling factors across blocks or multi-level scaling factors within subsets. MMA operations with a single- or multi-level scaled numeric format involve additional computational steps beyond those required by traditional formats. Conventional Tensor Cores primarily support operations with standard numeric formats but cannot efficiently handle the per-block scaling adjustments required by scaled formats. As a result, CUDA Cores should perform these scaling adjustments. This division of labor between Tensor and CUDA Cores increases instruction count and register file utilization. The added instruction count prolongs the execution time per thread, while higher register utilization further degrades energy efficiency.

To quantify the overhead of scaled numeric formats, we implement a GPU program that performs MMA operations on two 16×16 matrices using MXINT8, a single-level scaled numeric format. We compile the program with nvcc and analyze the executable with NVIDIA Nsight Compute to measure register usage and instruction count. The 16×16 matrix size matches the native MMA tile size of Tensor Cores. This choice also aligns with the recommendation of OCP, which suggests a block size of 32 for MX formats [2].

Figure 3 shows a stream of GPU instructions (PTX format) for an MMA operation with 16×16 tiles. During each MMA operation, the GPU loads shared scaling factors and applies them across multiple registers. For example, loading scaling factors into registers (e.g., R16, R17, R18, and R19) involves multiple steps, where each scaling factor adjusts the corresponding operands in matrices A and B before accumulation. This process introduces additional instructions, such as ‘mul’ and ‘mad’ (multiply-add), to scale each element individually. As a result, repeated multiplication and accumulation

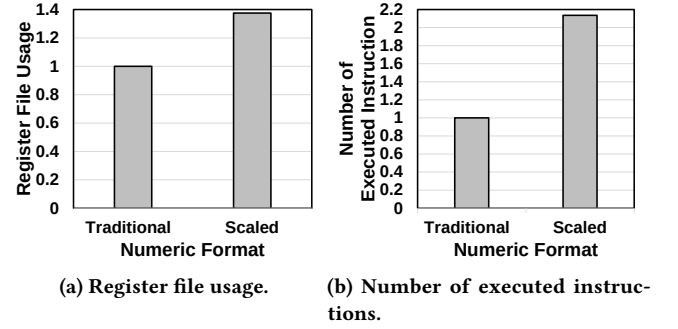


Figure 4: Comparison of register file usage and instruction count between traditional and scaled numeric formats. All results are normalized to INT8.

increase the instruction count, reflecting the inherent computational cost of handling scaled numeric formats. With multi-level scaled numeric formats, the GPU executes additional CUDA Core instructions to convert the data into a single-level format before MMA operations.

Through this detailed profiling, we also observe that the additional registers required to store shared scaling factors and intermediate values substantially contribute to the increased register footprint necessary for scaled numeric operations. We measure the register file usage and the number of executed instructions for traditional and scaled numeric formats in real DNN workloads. Specifically, we compare INT8, a traditional numeric format, with MX9, a two-level scaled numeric format. We implement the matrix multiplication operation in the first layer of ViT-Huge using each format and analyze the register usage with the nvcc compiler.

Figure 4 shows the experimental results. In our experiments, the register file usage for MX9 is $1.38\times$ higher than that of INT8. Excessive register usage per thread may reduce the number of threads that can execute concurrently on each streaming multiprocessor (SM) [34]. This reduction in executable threads can lead to performance degradation. Also, as shown in Figure 4b, the number of instructions executed for MX9 is $2.14\times$ higher than that of INT8, further degrading performance and efficiency. Thus, while scaled numeric formats improve arithmetic density, their increased register and instruction requirements may result in significant slowdown.

3 Avant-Garde

The key idea behind Avant-Garde is the “flattening” of scaled numeric formats into a predefined single-level scaled numeric format in microarchitecture to perform arithmetic operations on the flattened operands using custom Tensor Cores. Both single-level and multi-level scaled numeric formats follow a common flattening process. This process is straightforward for single-level formats, where one scaling factor is uniformly shared among all elements in the original and the flattened representation. In contrast, multi-level formats recursively apply scaling factors one level at a time to arrive at a flattened representation. The resulting flattened representation can be stored in memory and maintained in the register file, eliminating the need to flatten operands for subsequent computation. Flattening not only enables consistent computation patterns for

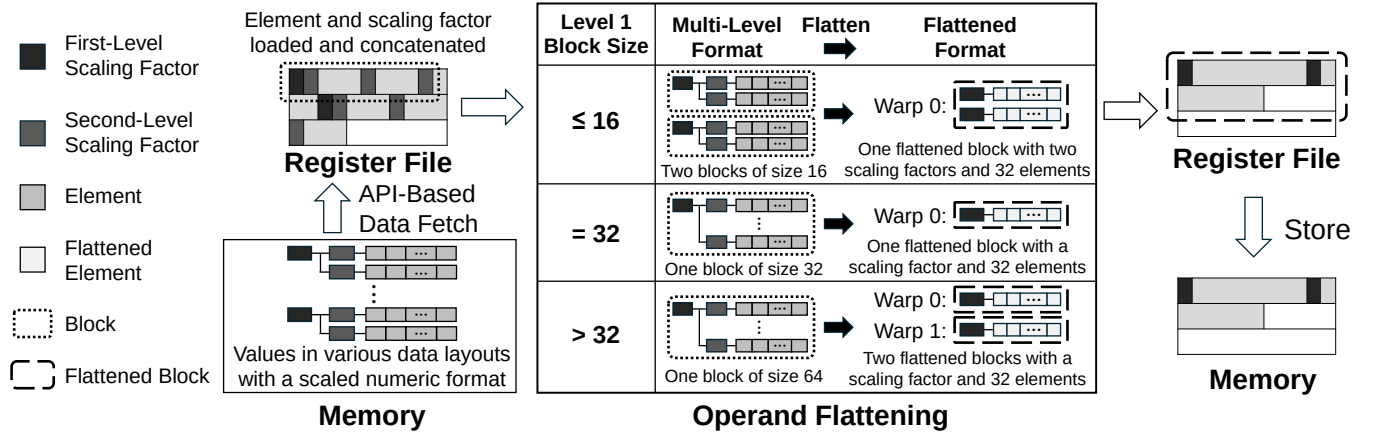


Figure 5: Operand flattening in Avant-Garde. Avant-Garde loads operands in scaled numeric formats into the register file, then flattens them into a flattened format before performing arithmetic operations in a Tensor Core. Avant-Garde maintains the operands in the flattened format in the register file and memory for subsequent operations.

DNN workloads but also aligns with the GPU execution model, ensuring efficient utilization of GPU resources.

Figure 5 illustrates the flattening process in Avant-Garde for two-level scaled numeric formats. Flattening results in a single-level format, with a *flattened block* of elements and a scaling factor. For two-level scaled numeric formats, flattening applies all second-level scaling factors to the elements while retaining the first-level scaling factor as is. As GPU execution is organized into warps (typically 32 threads/warp), we set the flattened block size to the GPU warp size and coalesce smaller blocks into a full warp register (typically 128 bytes) to maximize the utilization of resources. The figure shows that for small block sizes (< 16 elements), Avant-Garde first flattens each block, then coalesces as many blocks into a single flattened block while preserving their scaling factors until the number of elements reaches a maximum of 32. For large block sizes (> 32 elements), Avant-Garde slices the block into multiple flattened blocks, with each slice retaining the original scaling factor.

A key requirement for the flattening process is to give the application developer the flexibility to freely use scaled numeric formats allowing for denser storage and computation. Recent DNN research focuses on maximizing compression along multiple computational dimensions with mixed precision. Implementing the proposed flattening process for a wide spectrum of scaled numeric formats requires careful consideration of the data layout and storage. If not careful, flattening may lead to a low utilization of resources for formats with two or more scaling levels because applying the scaling factors results in scalar computation that may not align with the GPU’s single-instruction multiple-thread (SIMT) execution model.

To address this challenge, Avant-Garde introduces a novel GPU microarchitecture and provides an API for efficient data transformations. Avant-Garde integrates custom microarchitecture to handle scaling factors, supporting both existing and future scaled numeric formats without requiring hardware modifications. In addition, per-element formatting flexibility within a block can be expanded by adopting prior software-based solutions. For example, VitBit offers fine-grained control over bit allocation and scaling mechanisms

at the software level [20]. By combining Avant-Garde with the element format-aware software processing of VitBit, future scaled numeric formats can be accommodated with minimal hardware.

The Avant-Garde API provides a structured interface for programmers to retrieve data. It assumes that programmers understand the data layout and use the API to fetch elements and scaling factors accordingly. With this design, elements and scaling factors are stored side by side in the register file, supporting co-located access. The latter aligns with the flattening strategy and maintains optimized storage and data flow for high-performance DNN workloads.

3.1 Basic Pipeline

Figure 6 presents the anatomy of the Avant-Garde pipeline. To flatten operands, Avant-Garde adds a pipeline stage for *Operand Transformer*, a dedicated microarchitecture component between the operand read and execute stages in the baseline GPU pipeline. Avant-Garde adds instructions to use Operand Transformer to flatten operands from scaled numeric formats. Avant-Garde also includes instructions for the developer to specify the scaled numeric format parameters including the number of scaling levels, the block size and bitwidth of each element in scalar registers. The flattening instructions use these parameters with a pair of source and destination 128-byte warp registers to implement the transformation.

Avant-Garde’s Tensor Core also includes an MMA instruction that operates on flattened operands. To support MMA operations, Avant-Garde incorporates a custom Tensor Core enhanced to operate on the flattened format. Avant-Garde’s Tensor Core directly performs computation on operands represented with a flattened block with a scaling factor. Arithmetic involving both a scaling factor and a block elements is similar to exponent and mantissa management with a conventional floating point as scaling factors should be adjusted for both source and destination operands depending on the operation. For example, a multiplication of two operands requires addition of scaling factors and a pairwise multiplication of elements in each block, while addition requires adjust scaling factors to allow for pairwise addition of elements.

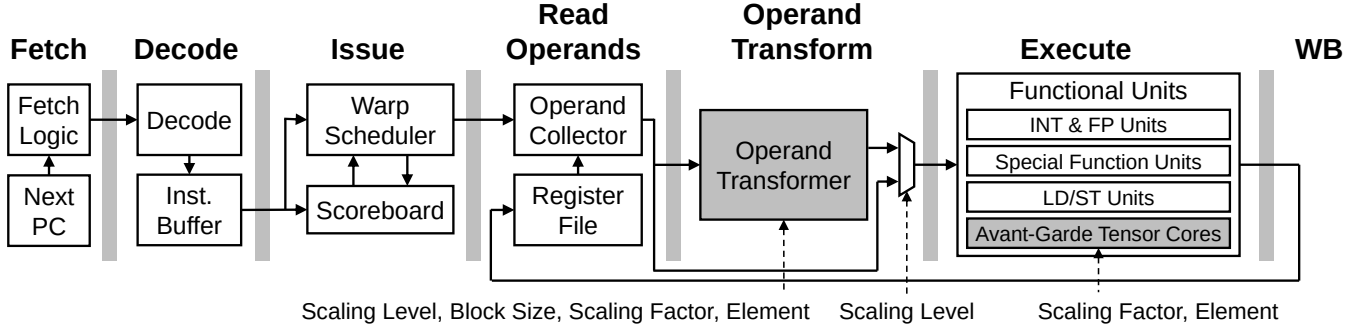


Figure 6: Basic Avant-Garde pipeline. The pipeline is derived from a conventional GPU with modified components highlighted in gray. The Operand Transform stage which flattens operands is selectively skipped if the operands do not use a scaled numeric format. Avant-Garde’s Tensor Core performs dot product operations on flattened operands.

Avant-Garde uses the GPU’s existing load/store instructions to transfer operands between the memory and register file. Operands are laid out contiguously in memory for any format including the conventional data types, scaled numeric formats and the flattened format. Tensor Cores in conventional GPUs have hard-wired connections that scatter elements from 128-byte warp registers to arithmetic units [46]. This microarchitecture supports operand fetching and supplying even if elements are not 4-byte aligned within a warp register. Leveraging this microarchitecture capability, Avant-Garde’s Tensor Cores interpret flattened blocks without 4-byte alignment constraints. This functionality is integrated into the register file arbitrator. For example, with the MX6 format, Avant-Garde requires only 192 bytes for a block, occupying two warp registers and leaving 64 bytes unused.

Finally, for all non-GEMM operations, Avant-Garde maintains operands in registers in the same manner as the baseline GPU described in Section 2.2. These operations require a source operand for a warp to be stored in an aligned warp register consisting of 32 four-byte registers. For instance, if a block in a scaled numeric format contains four-bit elements, each element is stored in a four-byte register, leaving the remaining 28 bits unused. Because operations other than GEMM computations represent only a small portion of the total workload, GPU kernels using non-GEMM operations do not significantly affect average register file utilization. Moreover, as discussed in Section 2.2, register pressure in conventional GPUs during computations with scaled numeric formats arises from additional registers used to temporarily store intermediate results for arithmetic operations. Avant-Garde mitigates this overhead through dedicated Operand Transformers and redesigned Tensor Cores. This design achieves significant improvements over conventional GPUs without increasing register file utilization.

3.2 Implementation & API

Figure 7 describes the microarchitecture of Operand Transformer. Its input, blocks in a scaled numeric format, comes from operand collectors connected to the register file. The outputs generated by Operand Transformer include one or more scaling factors (if there are multiple flattened blocks per warp) and sequences of elements, with each element associated with its respective scaling factor. Within the Operand Transformer, the warp operand formation

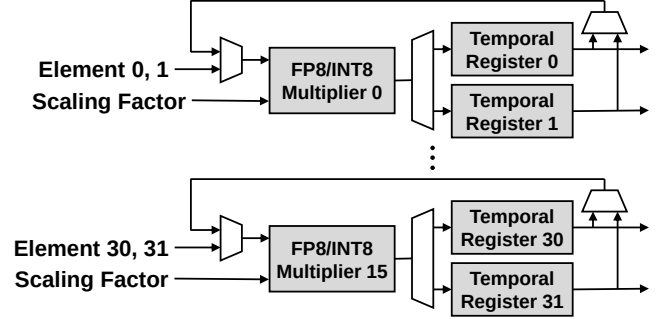


Figure 7: Operand Transformer. This unit incorporates 16 FP8/INT8 multipliers and thirty-two temporal registers. Each multiplier output is propagated to one of two temporal registers. To handle 32 elements, the unit reuses 16 multipliers twice.

controller assigns flattened blocks to warps based on the strategy shown in Figure 5. Each flattened block contains up to 32 elements and one or more associated scaling factors, aligned to the warp size. Small blocks may be coalesced into a warp, while large blocks may be split across multiple flattened blocks.

As operand flattening involves transforming multi-level formats, its latency depends on both the scaling level and the block size of the scaled numeric format. For a scaling level of N , Operand Transformer performs $2 \times (N - 1)$ iterations to flatten the multi-level format into a single-level one. The block size also influences the number of required iterations. Larger block sizes, such as those used in HBBP [14], reduce the frequency of transformation operations because multiple operands can be processed simultaneously. During each iteration, intermediate values generated by the flattening process are stored in temporary registers, each occupying 32 bytes.

Although Operand Transformer may perform multiple iterations for complex operand flattening scenarios, this overhead has minimal impact on overall performance. During DNN workload execution, operand flattening occurs as a preprocessing step rather than on a per-operation basis, minimizing its impact on execution time. For model weights, the transformation is applied once before inference

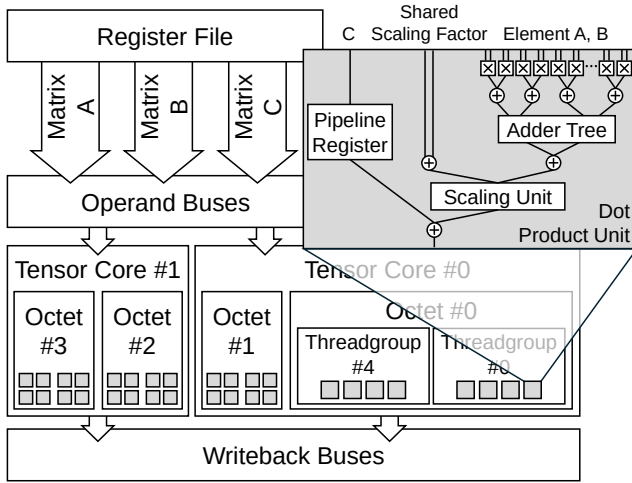


Figure 8: Avant-Garde Tensor Core. This unit incorporates an 8-bit fixed-point adder for combining the scaling factors of two operands and a scaling unit for multiplying the dot product result with the combined scaling factor before passing it to the accumulator.

or training to convert pre-stored multi-level representations into flattened formats, avoiding repeated transformations. For input data, operand flattening is applied at the beginning of computation and remains unchanged throughout execution. Activations are computed and retained in this flattened format, eliminating the need for additional flattening. Moreover, the latency introduced by operand flattening is often hidden by the interleaved warp execution of GPUs. As a result, Avant-Garde processes multi-level formats efficiently without degrading performance. We evaluate the impact of operand flattening latency across various scaled numeric formats. Our results demonstrate that Avant-Garde maintains performance improvements in these scenarios (details in Section 5.6).

Figure 8 describes the detailed microarchitecture of Avant-Garde’s Tensor Core. It incorporates two specialized microarchitecture units: an 8-bit fixed-point adder for combining scaling factors and a scaling unit that applies combined scaling factors to the results of dot products across elements. As scaling factors are encoded as exponents, the 8-bit fixed-point adder computes the sum of scaling factors from each operand.

Avant-Garde’s Tensor Cores send the combined scaling factor to a dedicated scaling unit, which performs a low-latency multiplication that applies the combined scaling factor to the dot product result before accumulation. The result of scaling unit then proceeds to the accumulator and continues through the rest of the pipeline. By embedding this operation within the core pipeline, Avant-Garde eliminates the need for separate scaling instructions, reducing register pressure and instruction count.

The Avant-Garde API extends the warp matrix multiplication and accumulation (WMMA) API to support predefined scaled numeric formats. Figure 9 illustrates a use case of the Avant-Garde API. In this example, the *main* function declares tensors and allocate corresponding memory space. The *compute_gemm_scaled* function

```
_global_
void compute_gemm_scaled(half *A, half *B, float *C,
                        int M, int N, int K, scaled *format) {
    wmma::fragment<wmma::matrix_a, M, N, K, format,
                    wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, M, N, K, format,
                    wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, M, N, K, format> acc_frag;
    ...
    wmma::load_matrix_sync(a_frag, A, M);
    wmma::load_matrix_sync(b_frag, B, K);
    wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
    wmma::store_matrix_sync(C, acc_frag, N, wmma::mem_row_major);
}

void inference(half *i, half *w, half *o, scaled *format) {
    ...
    compute_gemm_scaled<<...>>(A, B, C, M, K, N, format);
}

int main(int argc, char** argv) {
    scaled mx9;

    cudaMalloc((void**)&input, mx9.getSize() * INPUT_SIZE);
    cudaMalloc((void**)&weight, mx9.getSize() * WEIGHT_SIZE);
    cudaMemcpy(weight, weight_h, mx9.getSize() * WEIGHT_SIZE,
               cudaMemcpyHostToDevice);
    ...
    // Flatten multi level into single level.
    flatten(w_flattened, weight);

    while (runningInference) {
        cudaMemcpy(input, input_h, mx9.getSize() * INPUT_SIZE,
                   cudaMemcpyHostToDevice);
        flatten(i_flattened, input);
        inference(i_flattened, w_flattened, output, mx9);
    }
    ...
}
```

Figure 9: Avant-Garde API and use case. User can configure settings such as the scaling level and block size of the scaled format through the API. Once configurations for the scaled numeric format are specified, then GPU memory functions can exploit the specifications.

performs matrix multiplication using scaled numeric formats. These tensors are defined with MX9 format. The PTX or SASS interpreter in Avant-Garde identifies the target scaled numeric format through a predetermined variable, *scaled*, and assigns it to the relevant intermediate representations. The Avant-Garde API provides data formats for predefined scaled numeric formats, specifying scaling levels, block sizes, and the bit compositions of scaling factors and elements. For example, if a user selects MX9, the Avant-Garde API provides predefined information including its scaling level (2), the number of subsets per block (8), and the block size (16). Based on this information, Avant-Garde accesses data in a scaled numeric format and flattens it accordingly without requiring additional manual configuration.

To transform data from a scaled numeric format into flattened blocks, the Avant-Garde API provides the *flatten* function. In the example shown in Figure 9, this function takes pointers to tensors in the MX9 format. Avant-Garde applies the *flatten* function to generate flattened blocks, invoking it twice: once for the weights and once for the inputs.

Avant-Garde API in GPU kernels is compiled into a SASS instruction stream that includes newly defined instructions. To perform operand flattening, we define FLAT instruction, which transforms multi-level scaled numeric formats into a flattened format. Also, we define Flattened MMA (FMMA) instruction, an extension of the WMMA instruction for scaled numeric formats. For example, the *FMMA.16816.mx9.mx9* instruction performs matrix-matrix multiplication using flattened blocks generated from MX9 formats. The *16816* suffix denotes the tile dimensions used in the operation, where 16×16 represents the output tile size and 8×16 indicates the shape of the input operands. This suffix is the same as that used in existing WMMA instructions.

Users can specify a scaled numeric format while declaring fragments, enabling the extended *wmma* functions to correctly interpret and process the data. The *wmma :: mma_sync* function supports two modes of MMA operations. It can either (1) convert the Tensor Core’s floating-point result into a flattened format, or (2) retain it in floating point. Users can configure the mode depending on whether a subsequent layer in a DNN model requires flattened blocks or floating-point values. If *wmma :: matrix_a* (or *wmma :: matrix_b*) is selected, the result remains in floating point. In this case, the output of a single MMA operation on Avant-Garde’s Tensor Core is a floating-point value, which is directly written to the register file. In contrast, if *wmma :: accumulator* is selected, the floating-point results are converted into a flattened block once Avant-Garde’s Tensor Cores generate a predetermined number of them. The *wmma :: mma_sync* function additionally issues *FLAT* instructions, which take the floating-point results from the Tensor Core and transform it into the flattened format. In this way, Avant-Garde produces the outputs of MMA operations in the flattened format without incurring significant overhead.

During training, Avant-Garde performs unflattening operations that reconstruct the predefined original scaled numeric formats from flattened blocks. We design an “unflattening” API that converts the updated weights into the scaled numeric format and stores them in memory. The unflattening API first reorganizes the elements by splitting or coalescing flattened blocks to match the specified block size. For formats with two or more scaling levels, the API groups elements into subsets, selects appropriate scaling factors based on the maximum magnitude, and scales the elements accordingly. It recursively applies this scaling process to generate multiple levels of scaling factors until the format specification is fully satisfied. Finally, the API packs the scaling factors and elements and writes them back to memory.

The unflattening API leverages CUDA cores to perform data transformation. CUDA cores operate on operands in flattened format stored in the register file. Since each element in a flattened block is represented in fixed-point format, extracting the exponent and applying scaling operations can be implemented in a straightforward manner. In this manner, Avant-Garde transforms and stores flattened data in a scaled numeric format, thereby reducing memory usage while minimizing precision loss. As these operations are performed on CUDA cores, they introduce a long latency. However, since unflattening occurs infrequently, its overhead has minimal impact on overall GPU performance.

Table 1: Simulation configurations

GPU	NVIDIA Hopper (H100), 114 SMs max 64 warps/SM, max 32 block/SM
Scheduler	4 warp schedulers/SM, GTO
L1	192 KB
L2	40 MB, 32 way
Register File	256 KB, 8 banks per SM

Table 2: Numeric formats used in evaluation

Format	Scaling Level 1		Scaling Level 2		Element
	Block Size	Scaling Factor	Block Size	Scaling Factor	
HBFP	64	8 bits	N/A	N/A	8-bit fixed point
MX9	16	8 bits	2	1 bit	8-bit fixed point
MXFP8	32	8 bits	N/A	N/A	FP8

3.3 Silicon Overhead

We analyze the microarchitecture overhead introduced by the added components in Avant-Garde. We synthesize Operand Transformer and Avant-Garde’s Tensor Core using FreePDK 45nm technology to evaluate area and power overhead [44]. We set the area and power consumption of an H100 GPU as the baseline. Operand Transformer integrates logic for scaling factor alignment and flattened block creation. The Operand Transform stage incorporates 16 FP8/INT8 multipliers and temporary registers for intermediate value storage. This stage adds 1.2% area and 1.7% power overhead relative to a standard SM and introduces a latency impact of two cycles per warp due to iterative flattening for multi-level formats. Avant-Garde’s Tensor Core applies combined scaling factors to the output of the adder tree before accumulation. The redesigned Tensor Core integrates logic for shared scaling factors and a scaling unit to support scaled numeric formats, resulting in 3.9% area overhead and 3.1% power overhead compared to the standard Tensor Core.

The overall microarchitecture overhead integrating Operand Transformer and the redesigned Tensor Core amounts to roughly 1.4% in area and 1.2% in power compared to a conventional GPU pipeline. Of the 1.2% power overhead, leakage power contributes only 0.1% to the total power while dynamic power accounts for roughly 1.1%. This overhead is offset by significant gains in arithmetic density and computational efficiency enabled by Avant-Garde.

4 Methodology

We model Avant-Garde with Accel-Sim [21]. The detailed specifications of our baseline are described in Table 1. We use the NVIDIA H100 GPU as our baseline, which supports FP8 [33]. Aligning with NVIDIA’s approach, we assume that Avant-Garde supports FP8 in the same manner as the baseline. As Accel-Sim does not support FP8, we modify the simulator to compute a scaling factor so that FP8 operations execute with the same latency as INT8. As both formats have the same 8-bit data width, they exhibit identical memory access patterns and storage characteristics. We also model the

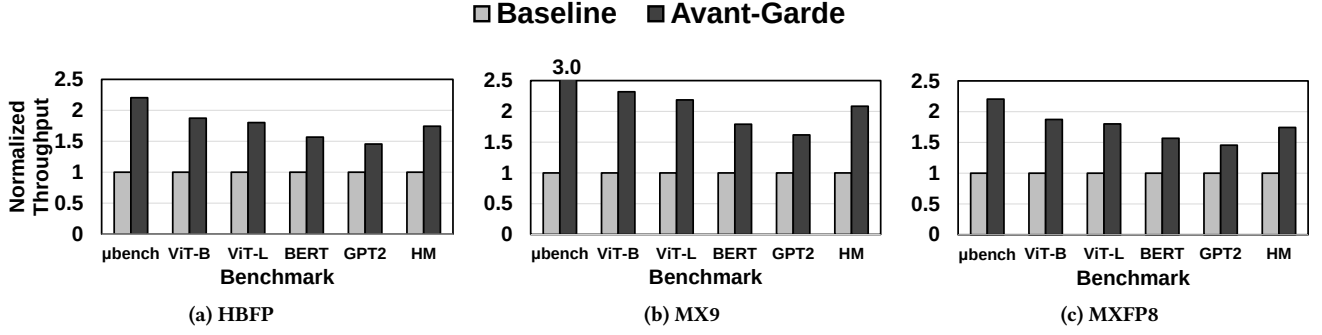


Figure 10: Throughput comparison. All results are normalized to the baseline. ViT-B and ViT-L represent ViT-Base and ViT-Large, respectively. HM represents the harmonic mean of all workloads.

Table 3: Benchmarks

Benchmark	Parameters	Dataset
Microbenchmark	1M	N/A
Vision Transformer Base (ViT-Base)	86M	ImageNet
Vision Transformer Large (ViT-Large)	307M	ImageNet
Bert-base-uncased (BERT)	110M	English Wikipedia Text
Generative Pre-trained Transformer 2 Small (GPT-2)	124M	English Wikipedia Text

latency and throughput of Tensor Cores and the microarchitecture to account for FP8 execution in our simulator. Note that the core behavior of Avant-Garde is orthogonal to the specific features of any particular GPU.

For power modeling, we extend AccelWattch to include FP8-specific power characteristics by scaling the power values of INT8 Tensor Core operations, given their similar compute units and memory access patterns. Also, we account for FP8 instruction-level power overhead based on our analysis in Section 3.3.

We evaluate the performance of Avant-Garde using three scaled numeric formats: HBFP, MX9, and MXFP8. For HBFP, we use an 8-bit mantissa and a block size of 64 [14]. We use MX-compliant formats provided by prior work and the OCP Specification, including MX9, MXFP8 [2, 42]. The detailed configuration of the scaling factor is provided in Table 2.

We list the benchmarks in Table 3. To evaluate the performance of Avant-Garde, we implement two types of workloads. We implement a matrix multiplication microbenchmark with 1M parameters. Although the microbenchmark cannot perform inference, it closely resembles a real DNN model, and the majority of its operations are MMA operations. Along with the microbenchmark, we use four DNN models: Vision Transformer (ViT-Base, ViT-Large) [7], BERT[6], and GPT-2 [38]. We use the ImageNet dataset [5] for ViT models, and English Wikipedia [11] for BERT and GPT2.

In the baseline, we implement a DNN model that handles the scaling factor in software to support the scaled numeric formats.

Using the WMMA API, we perform matrix multiplication on Tensor Cores and incorporate CUDA instructions to compute and apply the scaling factor. We then embed the functionalities of the Avant-Garde API in Accel-Sim and evaluate the GPU behaviors.

5 Evaluation

5.1 Performance Comparison

We first evaluate Avant-Garde’s throughput across all configurations, defining throughput as the number of operations per clock cycle. We run a microbenchmark and a single inference pass of each DNN model.

Figure 10 shows the experimental results. Avant-Garde achieves a throughput improvement ranging from $1.65\times$ to $1.93\times$ compared to the baseline, with an overall improvement of $1.74\times$ (harmonic mean). This gain is primarily due to a reduced instruction count. Tensor Cores of the baseline do not support applying a scaling factor to the dot product results. Therefore, while performing matrix multiplication with single-level formats on the baseline, additional instructions from CUDA Cores are required for every MMA operation on Tensor Cores. Avant-Garde processes scaled formats efficiently by applying the scaling factor directly to the dot product results and accumulating it within Avant-Garde’s Tensor Cores, without the need for additional instructions.

For both HBFP and MXFP8, Avant-Garde achieves a $1.66\times$ higher harmonic mean throughput than the baseline in DNN workloads. HBFP and MXFP8 leverage the INT8 and FP8 engines of Tensor Cores, respectively, as their elements are represented in INT8 and FP8 formats. On the H100 GPU, these two engines provide equivalent throughput, resulting in identical performance improvements across the two formats.

For MX9, Avant-Garde achieves a $1.93\times$ higher harmonic mean throughput than the baseline in DNN workloads. In the baseline, computing two-level formats requires additional software overhead. Avant-Garde eliminates this overhead by introducing the Operand Transform stage. By removing the overhead of two-level scaling format conversion and matrix multiplication on the flattened format, Avant-Garde achieves its highest throughput with MX9.

The throughput improvement of Avant-Garde slightly decreases as model size increases. For ViT-Base (ViT-B), the average performance improvement is 6% higher than for ViT-Large (ViT-L) across

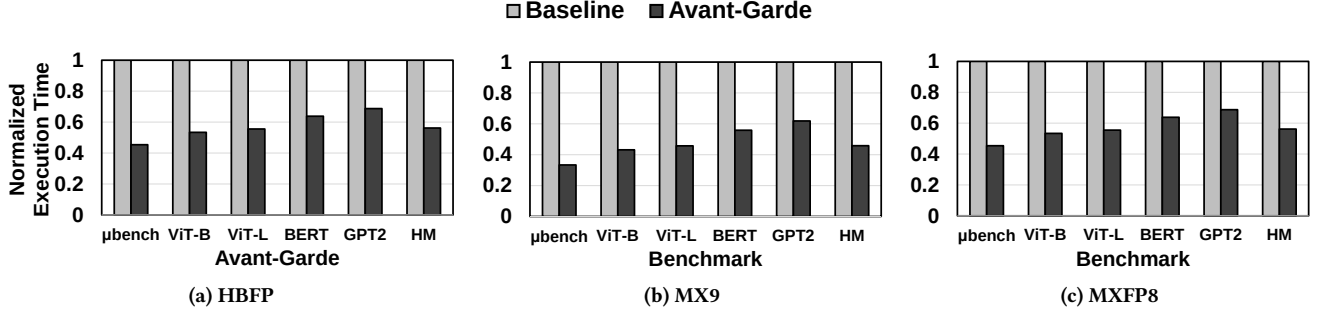


Figure 11: Execution time comparison. In DNN workloads, execution time corresponds to inference time. All results are normalized to the baseline. ViT-B and ViT-L represent ViT-Base and ViT-Large, respectively. HM represents the harmonic mean of all workloads.

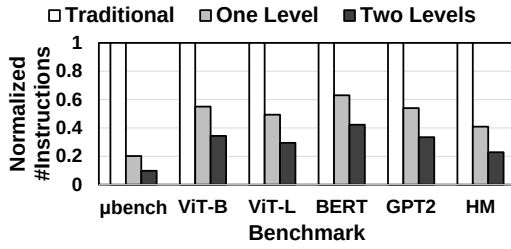


Figure 12: Instruction count comparison. All results are normalized to the baseline.

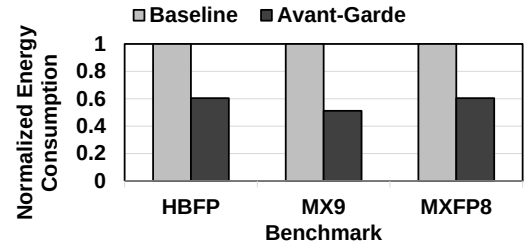


Figure 13: Energy consumption comparison while running DNN models with scaled numeric formats. All results are normalized to the baseline.

all three numeric formats. Larger models require more frequent memory accesses than small models, and the operand flattening mechanism may introduce additional accesses for scaling factors. This behavior could slightly offset the benefits of operand transformation, leading to diminishing returns in larger models.

5.2 Execution Time

We compare the execution time of all the configurations. Figure 11 shows the experimental results. In case of microbenchmark, the reported values indicate the inference time for a single request. The improvements in throughput directly translate to reduced execution time. In DNN models, Avant-Garde reduces the harmonic mean of DNN execution time by 44%. Also, as mentioned in Section 5, performance gains slightly diminish with increasing model size. In our experiments, Avant-Garde reduces the execution time of ViT-L by 48% while it reduces that of ViT-B by 51%.

Avant-Garde achieves a significant reduction in execution time for the microbenchmark, ranging from 55% to 67%. This reduction in execution time stems from Avant-Garde’s ability to efficiently mitigate the overhead introduced by MMA operations involving scaled numeric formats on Tensor Cores. Avant-Garde reduces the overhead associated with applying scaling factors in MMA operations and achieves its best performance in the microbenchmark.

5.3 Instruction Count Comparison

We measure the number of instructions executed by the baseline and Avant-Garde using the traditional, single-level, and two-level

formats. We use INT8 as the baseline traditional numeric format. Figure 12 presents the number of instructions executed by Avant-Garde. We observe that HBFP and MXFP8 exhibit similar trends in the instruction count. Therefore, we report their average under the label “One Level”. Likewise, the results for MX9 are denoted as “Two Levels” to maintain consistent representation across experiments. For the traditional format, the baseline and Avant-Garde execute the same number of instructions. However, in DNN workloads using the single-level and two-level formats, compared to the baseline, Avant-Garde reduces the number of executed instructions by 52.2% and 65.7%, respectively. This trend is consistently observed across all benchmarks.

By natively supporting scaled numeric formats and integrating Operand Transformer, Avant-Garde eliminates the need for software-based adjustments that rely on extra instructions executed by CUDA Cores. This reduction in instruction count is particularly pronounced for two-level formats compared to single-level formats, as Operand Transformer absorbs the second-level scaling factors into the element values. This process eliminates the need for separate instructions to handle the second-level scaling factors.

5.4 Energy Consumption

We measure the energy consumption of DNN models using scaled numeric formats. Figure 13 presents the harmonic mean of energy consumption across various DNN models. For all formats except FP8, Avant-Garde achieves a reduction in energy consumption. As analyzed in Section 3.3, Avant-Garde consumes 1.2% more power

Table 4: Accuracy comparison of FP32 and MX9 (Flattened and Not Flattened). For ViT, higher accuracy is better. For BERT and GPT-2, lower perplexity is better.

Model	FP32	MX9 (Flattened)	MX9 (Non-Flattened)
ViT-Base (Accuracy, %)	80.3	80.3	80.3
Bert (Perplexity)	10.09	10.11	10.10
GPT-2 (Perplexity)	17.48	17.49	17.49

than the baseline. Although the power consumption of Avant-Garde slightly increases compared to the baseline, the execution time is reduced for scaled formats, resulting in lower energy consumption. Avant-Garde achieves energy consumption reductions of 40%, 49%, and 40% for the HBFP, MX9, and MXFP8 formats, respectively. Among all formats, MX9 achieves the greatest reduction in energy consumption, as MX9 effectively removes instructions for handling two levels of scaling. Such behavior results in the most significant reduction in execution time.

For FP8-based workloads, the additional hardware in Avant-Garde remains inactive, and thus does not contribute to dynamic power consumption. Compared to the baseline, Avant-Garde consumes only 0.1% more energy, primarily due to a small amount of leakage power from the additional hardware resources, as discussed in Section 3.3. As such, Avant-Garde does not incur unnecessary energy consumption while handling FP8.

5.5 Accuracy Impact of Operand Flattening

For single-level formats, Avant-Garde can compute without requiring operand flattening, allowing lossless computation. As a result, it maintains the same accuracy as software implementations as discussed in Section 2.2. However, flattening a multi-level format into a single-level representation may introduce additional quantization error. To evaluate the impact of this flattening on model accuracy, we conduct functional simulations of inference using a two-level format within Avant-Garde and compare the accuracy results to FP32. Our methodology involves emulating Avant-Garde by modifying Microsoft’s MX emulator to capture the numerical behavior of the flattened format [31]. For comparison, we also measure the inference accuracy of the original (non-flattened) MX9 format using the same emulator.

Table 4 presents the accuracy results for FP32, flattened MX9, and original MX9 across ViT-Base, BERT, and GPT-2 models. All representations achieve nearly identical accuracies. The flattened MX9 format achieves the same accuracy as the non-flattened MX9 and shows less than a 0.2% difference compared to FP32. These results demonstrate that Avant-Garde maintains accuracy parity with FP32, demonstrating no significant degradation due to the flattening of the multi-level format. Operand transformation introduces no significant loss in precision, confirming that Avant-Garde is well-suited for accurate, low-bitwidth inference.

5.6 Sensitivity Study

We measure the execution time while running DNN workloads to evaluate the scalability of Avant-Garde across various scaling levels

and block sizes of scaled numeric formats. For the experiments, we use ViT-Large as the target model and set the baseline as inference using a single-level format with 8-bit elements and a block size of 32. All configurations are based on hypothetical numeric formats designed for evaluation purposes. To analyze the effect of scaling levels, we fix the block size at 32 and experiment with formats up to four scaling levels, using a subset size of 2. To evaluate sensitivity to block size, we maintain a single-level format while increasing the block size from 32 to 512.

In all cases, operand transformation remains a preprocessing step, completing flattening with a low latency compared to GEMM operations and requiring only minor memory accesses. The latency caused by these operations is often hidden by interleaved warp execution, minimizing its performance impact. As a result, operand transformation accounts for less than 1% of total execution time. As the overall performance across configurations shows minimal variation, we omit a plot for this analysis. The highest execution time increase occurs if the block size is 512, yet even in this case, the execution time increases by only 1.1% relative to the baseline. These results show that Avant-Garde continues to provide significant performance improvements over conventional GPUs, regardless of the specific scaling levels or block sizes used.

6 Related Work

Low-Precision and Scaled Numeric Formats. Quantization has been employed for inference to create lightweight DNN models that have parameters with narrow bitwidth formats. Using quantization methods, DNN model parameters can be compressed to formats as small as INT4 [1, 17, 52]. Not only for inference but also for training, a value-aware quantization technique has been proposed [36]. Prior work has explored various quantization methods for training, including specialized methods for INT8 [51] and Piecewise Fixed Point to mitigate quantization bias in low-precision training [28]. Prior work has introduced a mixed-precision training method that utilizes FP4 (1-bit sign, 3-bit exponent, and 0-bit mantissa) [45], representing the narrowest bitwidth format in DNN training.

Scaled numeric formats enhance efficiency by sharing scaling factors across blocks of values. HBFP combines the high accuracy of floating-point representations with the efficiency of block floating-point arithmetic [9, 14]. MSFP achieves 4× higher arithmetic density compared to INT8 by sharing exponents within bounding boxes, preserving accuracy with minimal fine-tuning [4, 41]. The subsequent microscaling builds upon shared microexponent by incorporating ultra-fine scaling factors at sub-block levels, enabling sub-8-bit precision for efficient training and inference [42]. These formats balance computational efficiency and accuracy, making them essential for DNNs.

Accelerators for Scaled Numeric Formats. Accelerators designed for scaled numeric formats are reshaping DNN performance paradigms. In particular, prior work has focused on DNN accelerators leveraging BFP formats, which fall under single-level scaled numeric formats.

The dynamic block size and precision scaling (DBPS) core exemplifies the potential of dynamic scaling by enabling adjustments to block size and precision during training [26]. Similarly, MSFP-based

architectures leverage shared exponents to optimize hardware efficiency, significantly improving arithmetic density and energy usage [41]. FAST introduces a novel processing element that performs dot product operations with variable precision at the layer level [50]. Bucket Getter addresses power and area overheads in BFP processing engines caused by floating-point adders [29]. MX-based accelerators extend these benefits by supporting ultra-low precision formats for diverse DNN tasks [42]. Avant-Garde builds on these concepts by introducing a GPU microarchitecture that flattens multi-level scaled numeric formats into single-level formats, maximizing arithmetic density and energy efficiency for both training and inference workloads.

GPU Software and Architectures for DNNs. Prior work proposed various methods that use GPU-based systems efficiently [3, 20, 27, 37, 47, 49]. Those methods are implemented in software and are supposed to utilize existing hardware resources in GPUs while running DNN workloads. Kim et al. proposed a GPU architecture for DNN workloads called Duplo [22]. Duplo includes a microarchitecture design inspired by hardware memoization [23]. With this scheme, Duplo could reduce the total computation count by exploiting fixed computation patterns and reusing the calculation results. While effective, prior work does not focus on reducing energy consumption further by designing a new numeric format. As such, they are orthogonal to Avant-Garde, which focuses on improving arithmetic density. We believe that the methods can be combined with Avant-Garde.

7 Conclusion

In this paper, we address the challenges of executing computations with scaled numeric formats in GPUs. These challenges stem from the lack of native GPU support for handling scaling factors, which requires software-based adjustments. This reliance increases register usage, introduces additional instructions during DNN workload execution. To address these issues, we propose Avant-Garde, an innovative GPU architecture designed specifically for efficient processing of scaled numeric formats. Avant-Garde introduces two primary innovations. The Avant-Garde microarchitecture incorporates a dedicated pipeline stage and enhanced Tensor Cores capable of natively handling scaling factors, streamlining MMA operations, and eliminating the software overhead typically associated with scaling factor adjustments. The Avant-Garde API simplifies developer interaction with the hardware, enabling easy integration while fully leveraging its capability. Avant-Garde significantly improves throughput and reduces execution time for DNN workloads compared to conventional GPUs.

Acknowledgments

The authors thank the anonymous reviewers, the members of ComSys at Korea University, and the members of PARSA at EPFL for their precious comments and feedback. This work was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (NRF-2022R1C1C1011021, RS-2024-00357037, RS-2025-00553645), a Microsoft Research PhD Fellowship, and “Unified Accelerators for Post-Moore Machine Learning” (200021_212757) from the Swiss National Science Foundation (SNSF). Babak Falsafi, Won Woo Ro, and Yunho Oh are the co-corresponding authors.

References

- [1] Ron Banner, Yury Nahshan, and Daniel Soudry. 2019. Post Training 4-Bit Quantization of Convolutional Networks for Rapid-Deployment. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NIPS)*. Curran Associates Inc., Red Hook, NY, USA, Article 714, 9 pages.
- [2] Tom Savell Ankit More Kyung-Nam Han Ritchie Zhao Mathew Hall Jasmine Klar Eric Chung Yuan Yu Michael Schulte Ralph Wittig Ian Bratt Nigel Stephens Jelena Milanovic John Brothers Pradeep Dubey Marius Cornea Alexander Heinecke Andres Rodriguez Martin Langhammer Summer Deng Maxim Naumov Paulius Micikevicius Michael Siu Bitu Darvish Rouhani, Nitin Garegrat and Colin Verrilli. 2023. OCP Microscaling Formats (MX) Specification. <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>.
- [3] Shubham Chaudhary, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, and Srinidhi Viswanatha. 2020. Balancing Efficiency and Fairness in Heterogeneous GPU Clusters for Deep Learning. In *Proceedings of the Fifteenth European Conference on Computer Systems (Heraklion, Greece) (EuroSys '20)*. Association for Computing Machinery, New York, NY, USA, Article 1, 16 pages.
- [4] Bitu Darvish Rouhani, Ritchie Zhao, Venmugil Elango, Rasoul Shafipour, Mathew Hall, Maral Mesmakhosroshahi, Ankit More, Levi Melnick, Maximilian Golub, Girish Varatkar, Lai Shao, Gaurav Kolhe, Dimitry Melts, Jasmine Klar, Renee L'Heureux, Matt Perry, Doug Burger, Eric Chung, Zhaoxia (Summer) Deng, Sam Naghshineh, Jongsoo Park, and Maxim Naumov. 2023. With Shared Microexponents, A Little Shifting Goes a Long Way. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (Orlando, FL, USA) (ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 83, 13 pages.
- [5] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 248–255.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). arXiv:1810.04805 <http://arxiv.org/abs/1810.04805>
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. arXiv:2010.11929 [cs.CV]
- [8] Mario Drumond, Louis Coulon, Arash Pourhabibi, Ahmet Caner Yüzügüler, Babak Falsafi, and Martin Jaggi. 2021. Equinox: Training (for Free) on a Custom Inference Accelerator. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (Virtual Event, Greece) (MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 421–433.
- [9] Mario Drumond, Tao Lin, Martin Jaggi, and Babak Falsafi. 2018. Training DNNs with Hybrid Block Floating Point. In *Advances in Neural Information Processing Systems (NIPS)*, Vol. 31. Curran Associates, Inc.
- [10] Luciano Floridi and Massimo Chiriatti. 2020. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines* 30, 4 (2020), 681–694.
- [11] Wikimedia Foundation. [n. d.]. Toggle the table of contents English Wikipedia. https://en.wikipedia.org/wiki/English_Wikipedia
- [12] Marcelo Gennari do Nascimento, Victor Adrian Prisacariu, Roger Fawcett, and Martin Langhammer. 2023. HyperBlock Floating Point: Generalised Quantization Scheme for Gradient and Inference Computation. In *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. 6353–6362.
- [13] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. EIE: Efficient Inference Engine on Compressed Deep Neural Network. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18–22, 2016*. IEEE Computer Society, 243–254.
- [14] Simla Burcu Harma, Ayan Chakraborty, Nicholas Sperry, Babak Falsafi, Martin Jaggi, and Yunho Oh. 2022. Accuracy Booster: Enabling 4-bit Fixed-point Arithmetic for DNN Training. arXiv:2211.10737 [cs.LG]
- [15] Intel. [n. d.]. Intel neural compressor. <https://intel.github.io/neural-compressor>. Accessed: 2025-02-16.
- [16] Intel. [n. d.]. OpenVINO: Open-source software toolkit for optimizing and deploying deep learning models. <https://github.com/openvinotoolkit/openvino>. Accessed: 2025-02-16.
- [17] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [18] Wonsuk Jang and Thierry Tambe. 2025. BlockDialect: Block-wise Fine-grained Mixed Format Quantization for Energy-Efficient LLM Inference. arXiv:2501.01144 [cs.CL] <https://arxiv.org/abs/2501.01144>
- [19] Anand Jayarajan, Jinliang Wei, Garth Gibson, Alexandra Fedorova, and Gennady Pekhimenko. 2019. Priority-based Parameter Propagation for Distributed DNN

- Training. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 1. 132–145.
- [20] Jaebom Jeon, Minseong Gil, Junsu Kim, Jaeyong Park, Gunjae Koo, Myung Kuk Yoon, and Yunho Oh. 2024. VitBit: Enhancing Embedded GPU Performance for AI Workloads through Register Operand Packing. In *Proceedings of the 53rd International Conference on Parallel Processing*. 1012–1021.
- [21] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. 2020. Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 473–486.
- [22] Hyeonjin Kim, Sungwoo Ahn, Yunho Oh, Bogil Kim, Won Woo Ro, and William J. Song. 2020. Duplo: Lifting Redundant Memory Accesses of Deep Neural Networks for GPU Tensor Cores. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 725–737.
- [23] Keonsoo Kim and Won Woo Ro. 2018. WIR: Warp Instruction Reuse to Minimize Repeated Computations in GPUs. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 389–402.
- [24] Jahyun Koo, Dahoon Park, Sangwoo Jung, and Jaeha Kung. 2024. OPAL: Outlier-Preserved Microscaling Quantization Accelerator for Generative Large Language Models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (San Francisco, CA, USA) (DAC '24)*. Association for Computing Machinery, New York, NY, USA, Article 259, 6 pages.
- [25] Urs Köster, Tristan Webb, Xin Wang, Marcel Nassar, Arjun K Bansal, William Constable, Oguz Elibol, Scott Gray, Stewart Hall, Luke Hornof, Amir Khosrowshahi, Carey Kloss, Ruby J Pai, and Naveen Rao. 2017. Flexpoint: An Adaptive Numerical Format for Efficient Training of Deep Neural Networks. In *Advances in Neural Information Processing Systems (NIPS)*, Vol. 30. Curran Associates, Inc.
- [26] Seunghyun Lee, Jeik Choi, Seockhwan Noh, Jahyun Koo, and Jaeha Kung. 2023. DBPS: Dynamic Block Size and Precision Scaling for Efficient DNN Training Supported by RISC-V ISA Extensions. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [27] Gangmuk Lim, Jeongseob Ahn, Wencong Xiao, Youngjin Kwon, and Myeongjae Jeon. 2021. Zico: Efficient GPU Memory Sharing for Concurrent DNN Training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 161–175. <https://www.usenix.org/conference/atc21/presentation/lim>
- [28] Chang Liu, Xishan Zhang, Rui Zhang, Ling Li, Shiyi Zhou, Di Huang, Zhen Li, Zidong Du, Shaoli Liu, and Tianshi Chen. 2022. Rethinking the Importance of Quantization Bias, Toward Full Low-Bit Training. *IEEE Transactions on Image Processing* 31 (2022), 7006–7019.
- [29] Yun-Chen Lo and Ren-Shuo Liu. 2023. Bucket Getter: A Bucket-based Processing Engine for Low-bit Block Floating Point (BFP) DNNs. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1002–1015.
- [30] Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoenybi, Michael Siu, and Hao Wu. 2022. FP8 Formats for Deep Learning. arXiv:2209.05433 [cs.LG]
- [31] Microsoft. 2024. MicroScaling Emulator. <https://github.com/microsoft/microscaling>.
- [32] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-Efficient Pipeline-Parallel DNN Training. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 7937–7947.
- [33] NVIDIA. 2020. NVIDIA A100 Tensor Core GPU Architecture. v1.0 (2020), 1–82.
- [34] Yunho Oh, Myung Kuk Yoon, William J. Song, and Won Woo Ro. 2018. FineReg: Fine-Grained Register File Management for Augmenting GPU Throughput. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 364–376.
- [35] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]
- [36] Eunhyeok Park, Sungjoo Yoo, and Peter Vajda. 2018. Value-Aware Quantization for Training and Inference of Neural Networks. In *Computer Vision – ECCV 2018: 15th European Conference, Munich, Germany, September 8–14, 2018, Proceedings, Part IV* (Munich, Germany). Springer-Verlag, Berlin, Heidelberg, 608–624.
- [37] Jay H. Park, Gyeongchan Yun, Chang M. Yi, Nguyen T. Nguyen, Seungmin Lee, Jaesik Choi, Sam H. Noh, and Young ri Choi. 2020. HetPipe: Enabling Large DNN Training on (Whimpy) Heterogeneous GPU Clusters through Integration of Pipelined Model Parallelism and Data Parallelism. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 307–321. <https://www.usenix.org/conference/atc20/presentation/park>
- [38] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2018. Language Models are Unsupervised Multitask Learners. (2018).
- [39] Akshat Ramachandran, Souvik Kundu, and Tushar Krishna. 2024. MicroScopiQ: Accelerating Foundational Models through Outlier-Aware Microscaling Quantization. arXiv:2411.05282 [cs.AR] <https://arxiv.org/abs/2411.05282>
- [40] MIT Technology Review. 2023. Multi-die systems define the future of semiconductors. <https://www.technologyreview.com/2023/03/31/1070527/multi-die-systems-define-the-future-of-semiconductors/>. Accessed: 2023-03-31.
- [41] Bitar Rouhani, Daniel Lo, Ritchie Zhao, Ming Liu, Jeremy Fowers, Kalin Ovtcharov, Anna Vinogradsky, Sarah Massengill, Lita Yang, Ray Bittner, Alessandro Forin, Haishan Zhu, Taesik Na, Prerak Patel, Shuai Che, Lok Chand Koppaka, Xia Song, Subhojit Som, Kaustav Das, Saurabh Tiwary, Steve Reinhardt, Sitaram Lanka, Eric Chung, and Doug Burger. 2020. Pushing the Limits of Narrow Precision Inference at Cloud Scale with Microsoft Floating Point. In *NeurIPS 2020*. ACM.
- [42] Bitar Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stolic Dusan, Venmugil Elango, Maximilian Golub, Alexander Heinecke, Phil James-Roxby, Dharmesh Jani, Gaurav Kolhe, Martin Langhammer, Ada Li, Levi Melnick, Maral Mesmahkoshroshahi, Andres Rodriguez, Michael Schulte, Rasoul Shafipour, Lei Shao, Michael Siu, Pradeep Dubey, Paulius Micikevicius, Maxim Naumov, Colin Verrilli, Ralph Wittig, Doug Burger, and Eric Chung. 2023. Microscaling Data Formats for Deep Learning. arXiv:2310.10537 [cs.LG]
- [43] Sayeh Sharify, Utkarsh Saxena, Zifei Xu, Wanzin Yazar, Ilya Soloveychik, and Xin Wang. 2024. Post Training Quantization of Large Language Models with Microscaling Formats. In *Proceedings of The 4th NeurIPS Efficient Natural Language and Speech Processing Workshop (Proceedings of Machine Learning Research, Vol. 262)*. Mehdi Rezagholizadeh, Peyman Passban, Soheila Samiee, Vahid Partovi Nia, Yu Cheng, Yue Deng, Qun Liu, and Boxing Chen (Eds.). PMLR, 241–258. <https://proceedings.mlr.press/v262/sharify24a.html>
- [44] James E. Stine, Ivan Castellanos, Michael Wood, Jeff Henson, Fred Love, W. Rhett Davis, Paul D. Franzon, Michael Bucher, Sunil Basavarajaiah, Julie Oh, and Ravi Jenkal. 2007. FreePDK: An Open-Source Variation-Aware Design Kit. In *2007 IEEE International Conference on Microelectronic Systems Education (MSE'07)*. 173–174.
- [45] Xiao Sun, Naigang Wang, Chia-Yu Chen, Jiamin Ni, Ankur Agrawal, Xiaodong Cui, Swagath Venkataramani, Kaoutar El Maghraoui, Vijayalakshmi (Viji) Srinivasan, and Kailash Gopalakrishnan. 2020. Ultra-Low Precision 4-bit Training of Deep Neural Networks. In *Advances in Neural Information Processing Systems (NIPS)*, Vol. 33. Curran Associates, Inc., 1796–1807.
- [46] Seunghwan Sung, Sujin Hur, Sungwoo Kim, Dongho Ha, Yunho Oh, and Won Woo Ro. 2023. MAD MAce: Supporting Multiply-Add Operations for Democratizing Matrix-Multiplication Accelerators. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (Toronto, ON, Canada) (MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 367–379.
- [47] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*. 41–53.
- [48] Yuke Wang, Boyuan Feng, and Yufei Ding. 2022. QGTC: Accelerating Quantized Graph Neural Networks via GPU Tensor Core. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (Seoul, Republic of Korea) (PPoPP '22)*. Association for Computing Machinery, New York, NY, USA, 107–119.
- [49] Jianbang Yang, Dahai Tang, Xiaoni Song, Lei Wang, Qiang Yin, Rong Chen, Wenyan Yu, and Jingren Zhou. 2022. GNNLab: A Factored System for Sample-Based GNN Training over GPUs. In *Proceedings of the Seventeenth European Conference on Computer Systems (Rennes, France) (EuroSys '22)*. Association for Computing Machinery, New York, NY, USA, 417–434.
- [50] Sai Qian Zhang, Bradley McDanel, and HT Kung. 2022. Fast: Dnn training under variable precision block floating point with stochastic rounding. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 846–860.
- [51] Kang Zhao, Sida Huang, Pan Pan, Yinghan Li, Yingya Zhang, Zhenyu Gu, and Yinghui Xu. 2021. Distribution Adaptive INT8 Quantization for Training CNNs. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 4 (May 2021), 3483–3491.
- [52] Aojun Zhou, Anbang Yao, Yiwen Guo, Lin Xu, and Yurong Chen. 2017. Incremental Network Quantization: Towards Lossless CNNs with Low-precision Weights. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*.